



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Practica 01: Intro a la programación multihilos en Java

Anaya Pérez Ulises Josué
Ortiz Amaya Bruno Fernando

PROFESORA

Gilde Valeria Rodríguez Jiménez

ASIGNATURA

Computación concurrente

1. **Lee lo siguiente** <https://www.evanjones.ca/software/threading-linus-msg.html> y comparte en máximo 4 líneas de computadora a qué se refiere Linus Torvalds con un contexto de ejecución y cómo se relaciona con la definición en la sección 1 de esta práctica.

Respuesta:

Linus Torvalds define un contexto de ejecución como la totalidad del estado necesario para que una tarea se ejecute, incluyendo el estado de la CPU, la MMU, permisos y estados de comunicación, dado que Linus enfatiza que no existe una diferencia entre **hilo** y **proceso**, esta forma de pensar se relaciona con las definiciones dadas en la sección 1.

2. **¿Cuántos hilos tiene disponibles tu computadora?** Ejecuta `Runtime.getRuntime().availableProcessors()`, si son más de uno en el equipo escriban el de cada uno.

- **Compu Ulises:**

Para esta parte creamos un script en java

```
~/Escritorio/8vo/Concurrente/practicass/p1
> code NumeroHilos.java
```

Con el siguiente código:

```
public class NumeroHilos {
    public static void main(String[] args) {
        int numHilos = Runtime.getRuntime().availableProcessors();
        System.out.println("Hilos en la computadora de Uli: " + numHilos);
    }
}
```

Ahora lo ejecutamos:

Entonces podemos ver que esta computadora tiene 8

```
~/Escritorio/8vo/Concurrente/practicass/p1
> javac NumeroHilos.java
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true

~/Escritorio/8vo/Concurrente/practicass/p1
> java NumeroHilos
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
Hilos en la computadora de Uli: 8
```

- **Compu Bruno:**

Con el siguiente código podemos ver cuantos hilos tiene la computadora:

```
1 public class Hilo {
2     public static void main(String[] args) {
3         System.out.println(Runtime.getRuntime().availableProcessors());
4     }
5 }
6
```

Ejecutando el código el resultado es el siguiente:

```
E:\Archivos\Codigo\Practicass-Concurrente2025-2>java Hilo.class
Error: Could not find or load main class Hilo.class
Caused by: java.lang.ClassNotFoundException: Hilo.class

E:\Archivos\Codigo\Practicass-Concurrente2025-2>java Hilo
12
```

Sabemos que la computadora de Bruno tiene 12 hilos

3. **Revisa el programa Determinante concurrente y responde** ¿Cuánto tiempo tarda en ejecutarse? Se realizaron varias ejecuciones donde el tiempo de ejecución varia con cada ejecución del programa, como se muestra en la siguiente imagen:

```
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 998100ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 840500ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 856700ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 788300ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 832700ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 1063600ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 970900ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 841500ns, result: -18

E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteConcurrente
Program took 1010300ns, result: -18
```

4. **El programa Determinante concurrente está implementado extendiendo la clase Thread.** Implementa el programa utilizando la interfaz Runnable.

Para implementar esto, lo haremos como vimos durante la clase del laboratorio Es bastante similar pero cambia porque en este script utilizamos la interfaz Runnable y ademas justo por esto mismo la forma en que creamos los hilos es diferente y tambien tenemos que sobrescribir el método run. Las partes que cambian en el código son las siguientes:

```
public class DeterminanteConcurrenteRunnable implements Runnable {
    static int determinante;
    static int n_prueba = 3;
    static int matriz_prueba[][] = { { 1, 2, 2 }, { 1, 0, -2 }, { 3, -1, 1 } };
```

```
public static int determinanteMatriz3x3(int matriz[][], int n_prueba) {
    int result = 0;
    DeterminanteConcurrenteRunnable t1 = new DeterminanteConcurrenteRunnable(matriz[0][0], matriz[1][1], matriz[2][2]);
    DeterminanteConcurrenteRunnable t2 = new DeterminanteConcurrenteRunnable(matriz[1][0], matriz[2][1], matriz[0][2]);
    DeterminanteConcurrenteRunnable t3 = new DeterminanteConcurrenteRunnable(matriz[2][0], matriz[0][1], matriz[1][2]);
    DeterminanteConcurrenteRunnable t4 = new DeterminanteConcurrenteRunnable(matriz[2][0], matriz[1][1], matriz[0][2]);
    DeterminanteConcurrenteRunnable t5 = new DeterminanteConcurrenteRunnable(matriz[1][0], matriz[0][1], matriz[2][2]);
    DeterminanteConcurrenteRunnable t6 = new DeterminanteConcurrenteRunnable(matriz[0][0], matriz[2][1], matriz[1][2]);
    Thread thr1 = new Thread(t1);
    Thread thr2 = new Thread(t2);
    Thread thr3 = new Thread(t3);
    Thread thr4 = new Thread(t4);
    Thread thr5 = new Thread(t5);
    Thread thr6 = new Thread(t6);
```

Método run:

```
@Override
public void run() {
    this.partial = this.num1 * this.num2 * this.num3;
}
```

El resto del código es igual, finalmente lo ejecutamos para ver el tiempo que tarda

```
~/E/8vo/Concurrente/FC_CConcurrente/Programas_P1/unam.fc.concurrent.practical on main !2 ?3
> java -cp bin unam.fc.concurrent.practica1.DeterminanteConcurrenteRunnable

Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
Program took 1844623ns, result: -18
```

5. **Implementa el programa Determinante concurrente de forma secuencial.** El código de la implementación se encuentra en el archivo **DeterminanteSecuencial.java**, de igual forma se comparte una captura de pantalla con la implementación realizada:

```
J DeterminanteSecuencial.java > DeterminanteSecuencial
1 public class DeterminanteSecuencial {
2     static int determinante;
3     static int n_prueba = 3;
4     static int matriz_prueba[][] = { { 1, 2, 2 }, { 1, 0, -2 }, { 3, -1, 1 } };
5     int num1, num2, num3, partial;
6
7     public DeterminanteSecuencial(int num1, int num2, int num3) {
8         this.num1 = num1;
9         this.num2 = num2;
10        this.num3 = num3;
11    }
12
13    public static int determinanteMatriz3x3(int matriz[][], int n_prueba) {
14        int term1 = matriz[0][0] * matriz[1][1] * matriz[2][2];
15        int term2 = matriz[1][0] * matriz[2][1] * matriz[0][2];
16        int term3 = matriz[2][0] * matriz[0][1] * matriz[1][2];
17        int term4 = matriz[2][0] * matriz[1][1] * matriz[0][2];
18        int term5 = matriz[1][0] * matriz[0][1] * matriz[2][2];
19        int term6 = matriz[0][0] * matriz[2][1] * matriz[1][2];
20
21        return (term1 + term2 + term3) - (term4 + term5 + term6);
22    }
23
24    Run | Debug
25    public static void main(String[] args) {
26        long startTime = System.nanoTime();
27        determinante = determinanteMatriz3x3(matriz_prueba, n_prueba);
28        long endTime = System.nanoTime();
29
30        System.out.println("Program took " + (endTime - startTime) + "ns, result: " + determinante);
31    }
32 }
```

Resultado de la ejecución de **DeterminanteSecuencial.java**:

```
E:\Archivos\Codigo\Practicas-Concurrente2025-2>javac DeterminanteSecuencial.java
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 3400ns, result: -18
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 3400ns, result: -18
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 3301ns, result: -18
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 4600ns, result: -18
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 3400ns, result: -18
E:\Archivos\Codigo\Practicas-Concurrente2025-2>java DeterminanteSecuencial
Program took 3400ns, result: -18
```

6. **Implementa del programa Determinante concurrente para dos hilos (en vez de seis).**
Ahora para implementar el programa con solo 2 hilos hicimos los siguientes cambios.

```
public DeterminanteConcurrenteDosHilos(int[][] valores) {
    this.valores = valores;
}

public static int determinanteMatriz3x3(int matriz[]) {
    int result = 0;

    int[][] valores1 = {
        { matriz[0][0], matriz[1][1], matriz[2][2] },
        { matriz[1][0], matriz[2][1], matriz[0][2] },
        { matriz[2][0], matriz[0][1], matriz[1][2] }
    };

    int[][] valores2 = {
        { matriz[2][0], matriz[1][1], matriz[0][2] },
        { matriz[1][0], matriz[0][1], matriz[2][2] },
        { matriz[0][0], matriz[2][1], matriz[1][2] }
    };

    DeterminanteConcurrenteDosHilos thr1 = new DeterminanteConcurrenteDosHilos(valores1);
    DeterminanteConcurrenteDosHilos thr2 = new DeterminanteConcurrenteDosHilos(valores2);

    thr1.start();
    thr2.start();

    try {
        thr1.join();
        thr2.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    result = thr1.partial - thr2.partial;
}
```

```
@Override
public void run() {
    this.partial = (valores[0][0] * valores[0][1] * valores[0][2]) +
        (valores[1][0] * valores[1][1] * valores[1][2]) +
        (valores[2][0] * valores[2][1] * valores[2][2]);
}

Run | Debug
public static void main(String[] args) {
    long startTime = System.nanoTime();
    determinante = determinanteMatriz3x3(matriz_prueba);
    long endTime = System.nanoTime();

    System.out.println("Program took " +
        (endTime - startTime) + "ns, result: " + determinante);
}
```

7. **Compara las 3 implementaciones:** el programa Determinante concurrente para dos hilos, para seis hilos y el programa secuencial. Responde: ¿A qué se debe el orden en el que se ordenan los tiempos de ejecución de cada programa? Al comparar los tiempos de ejecución nos percatamos que el orden de menor tiempo a mayor tiempo es el siguiente:

1. Secuencial
2. Dos hilos
3. Seis hilos

Esto se debe a que la implementación secuencial no sufre una sobrecarga al administrar los demás hilos, permitiendo que el flujo de ejecución se centre en la resolución del determinante de la matriz; la implementación con dos hilos puede mejorar ligeramente el rendimiento en sistemas con múltiples núcleos, pero la diferencia no es significativa debido a la baja complejidad del cálculo y la versión con seis hilos introduce aún más sobrecarga, lo que puede hacerla más lenta, especialmente si el número de núcleos es insuficiente.

8. Si utilizas la Ley de Amdahl entre el programa Determinante concurrente para dos hilos y el programa secuencial. ¿El resultado es mayor o menor a 1? ¿Por qué?

Recordemos lo que nos dice la ley de Amdahl:

$$S_p = \frac{1}{(1 - P) + \frac{P}{N}}$$

En el caso de este programa estamos usando 2 hilos, por lo que $N = 2$, en este programa solo estamos calculando el determinante de una matriz de 3×3 , por lo que no son tantas operaciones.

Sustituyendo $N = 2$ en la ecuación:

$$S_2 = \frac{1}{(1 - P) + \frac{P}{2}}$$

En este caso el resultado seria menor a 1, debido a que para sacar el determinante de una matriz son pocas operaciones. Las operaciones del programa son pocas, entonces si vemos las ejecuciones de ambos programas:

```
> java -cp bin unam.fc.concurrent.practica1.DeterminanteConcurrenteDosHilos
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
Program took 660854ns, result: -18
```

```
~/E/8vo/Concurrente/FC_CConcurrente/Programas_P1/unam.fc.concurrent.practica1.DeterminanteConcurrente
> java -cp bin unam.fc.concurrent.practica1.DeterminanteConcurrente
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
Program took 1175420ns, result: -18
```

Vemos que el programa concurrente con 2 hilos es más lento que el secuencial esto por el costo de sincronización y creación de hilos. Entonces nos convendría mas el uso de hilos si fueran mas operaciones.

9. **Describe con tus propias palabras en máximo dos líneas para qué sirve el método `join()`.** Si no utilizas el método `join()` en Determinante Concurrente, ¿sigue funcionando? El método `join()` pausa la ejecución del hilo principal del programa hasta que el hilo que invoco el método termine su ejecución, de esta forma se asegura que los datos manipulados por los hilos sean correctos.

Al no utilizar `join()`, el programa puede realizar su ejecución, sin embargo el resultado no sera el esperado o variara en cada ejecución, ya que el hilo principal podría acceder a los valores de **partial** antes de que los hilos secundarios los calculen.

10. Descripciones de los programas:

- DeterminanteConcurrenteRunnable.java: calcula la determinate de una matriz creando seis hilos atravez de la interfaz Runnable de Java.
- DeterminanteConcurrenteDosHilos.java: calcula la determinante de una matriz creando dos hilos de ejecucion para calcularla.
- DeterminanteSecuencial.java: calcula la determinate de una matriz de manera secuencial.

La version utilizada de Java fue **21.0.6 LTS**